Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

07
순환 신경망

문장은 단어가 순서대로 이어진 데이터입니다. 앞에서 읽은 내용을 기억하며 다음 단어를 읽는 신경망, RNN의 이론을 세우고 실제 문장과 다음 글자 예측에 적용해 봅니다.

오늘 다루는 것

1. 순환 신경망(RNN): 시퀀스 데이터, 은닉 상태, 입출력 구성, 깊은·양방향 RNN
2. LSTM과 GRU: 장기 의존성 문제와 게이트
3. 문자 단위 RNN(Char RNN): 다음 글자 맞히기

끝나고 나면 할 수 있는 것

1. 순서가 있는 데이터에 피드 포워드 신경망 대신 RNN을 쓰는 이유를 설명할 수 있다.
2. 은닉 상태의 갱신식 $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$를 읽고, 문장을 읽는 과정에 적용할 수 있다.
3. 바닐라 RNN의 장기 의존성 문제와 LSTM이 이를 해결하는 방법을 게이트로 설명할 수 있다.

2026년 10월 7일 수요일 (10/7 자료 2/3)
발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 막힌 질문은 디스코드에 남겨 주세요.

07

순환 신경망

"나는 밥을 먹었다"와 "밥을 나는 먹었다"는 같은 단어로 이루어졌지만, 단어의 **순서**가 의미를 만듭니다. 그리고 "먹었다"를 이해하려면 앞에 나온 "밥을"을 **기억**하고 있어야 합니다.
지금까지의 신경망(다층 퍼셉트론, CNN)은 입력을 한꺼번에 받아 한 번 계산하고 끝나서, 순서도 기억도 없습니다. 순환 신경망(RNN)은 단어를 하나씩 읽으면서 지금까지의 기억을 다음 단계로 넘겨 이 문제를 풉니다.

왜 배우나: 오늘의 학습 경로



문장은 순서가 있다
시퀀스 데이터

기억을 넘기며 한 단어씩 읽는다
RNN, 은닉 상태

길어지면 앞 기억이 사라진다
장기 의존성 문제

기억을 골라 지킨다
LSTM, GRU

다음 글자 맞히기
Char RNN

각 개념은 먼저 정의로 세우고, "문장을 한 단어씩 읽기"와 "휴대폰 키보드의 다음 글자 추천" 상황에 적용해 확인합니다. "다음 글자(단어)를 맞힌다"는 생각은 10장의 언어 모델, 나아가 GPT로 이어집니다.

07-01 순환 신경망(RNN)

핵심 한 줄

RNN은 같은 셀을 시점마다 반복해서 쓰면서, 이전 시점의 은닉 상태(기억)를 현재 입력과 함께 계산해 다음 시점으로 넘기는 신경망이다.

왜 배우나

자연어 처리의 데이터는 거의 모두 순서가 있는 데이터(문장, 대화, 음성)입니다. 순서와 앞 내용을 반영하지 못하면 "밥을 먹었다"와 "밥이 먹었다"의 차이도 구분할 수 없습니다. RNN은 이를 다루는 가장 기본적인 신경망이고, 07-02의 LSTM과 16장의 seq2seq, 17장의 어텐션이 모두 이 구조에서 출발합니다.

1) 시퀀스 데이터와 피드 포워드 신경망의 한계

정의

시퀀스(sequence) 데이터는 원소의 **순서가 의미를 가지는** 데이터입니다. 문장(단어의 순서), 음성, 주가처럼 시간 순서가 있는 데이터가 여기에 속합니다. 시퀀스에서 몇 번째 원소인지를 시점(time step) t로 나타냅니다.

- 피드 포워드 신경망(feed-forward): 입력에서 출력 방향으로 한 번만 계산하는 신경망(다층 퍼셉트론, CNN). 입력 크기가 고정되어야 하고, 이전 입력에 대한 기억이 없습니다.
- 그래서 시퀀스에는 맞지 않습니다. 문장마다 길이가 다르고, 뒤의 단어를 이해하려면 앞의 단어를 기억해야 하기 때문입니다.
- 순환 신경망(RNN, Recurrent Neural Network)은 출력의 일부를 **다음 시점의 입력으로 다시 보내는(순환)** 신경망입니다. 길이에 상관없이 원소를 하나씩 처리하며, 지금까지의 내용을 기억으로 요약해 들고 갑니다.

예시 적용

"나는 오늘 밥을 먹었다"는 시점 4개짜리 시퀀스입니다(시점 1 = "나는", …, 시점 4 = "먹었다"). 다층 퍼셉트론에 넣으려면 단어 4개를 한꺼번에 붙여 넣어야 하고, 단어가 3개나 5개인 문장은 다른 모델이 필요합니다. RNN은 같은 셀 하나로 길이 1, 2, 4인 문장을 모두 처리합니다(실습 2의 3번).

2) 메모리 셀과 은닉 상태

정의

RNN에서 계산을 하는 단위를 셀(cell) 또는 메모리 셀이라고 합니다. 셀이 시점 t에서 다음 시점으로 넘기는 값을 은닉 상태(hidden state) h_t라고 하며, 지금까지 읽은 내용의 요약, 즉 기억입니다.

$$h_t = \tanh\left(W_x\, x_t + W_h\, h_{t-1} + b\right)$$

- x_t: 현재 시점의 입력, h_{t-1}: 직전 시점의 은닉 상태, W_x, W_h, b: 학습할 가중치와 편향
- 새 기억은 **현재 입력**과 **이전 기억**을 섞어 만듭니다. 처음 시점의 h_0는 보통 0입니다.
- **모든 시점이 같은 W_x, W_h, b를 공유**합니다. 그래서 길이가 달라도 같은 셀로 처리할 수 있습니다.
- 출력이 필요하면 시점마다 $y_t = f(W_y\, h_t + b_y)$처럼 은닉 상태로 계산합니다.

예시 적용

RNN을 시간 순서대로 펼쳐 그리면, 같은 셀이 단어마다 반복되고 빨간 화살표로 기억이 전달됩니다.

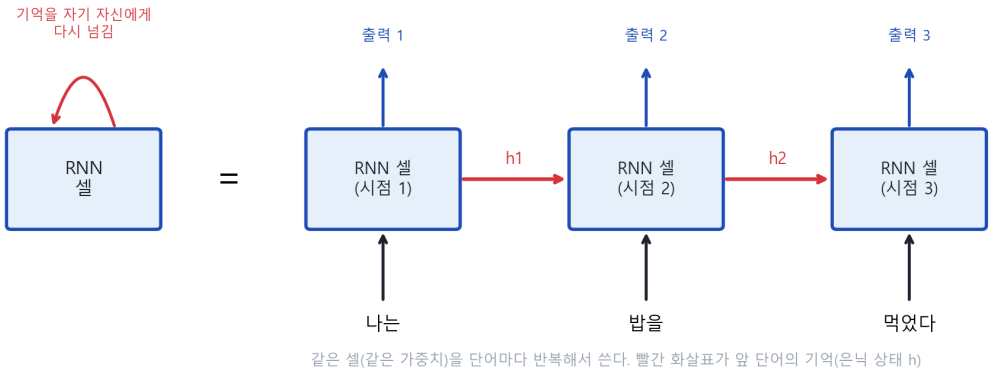


그림 1 RNN을 시간 순서대로 펼친 모습. 같은 셀(같은 가중치)이 단어마다 반복되고, 은닉 상태 h가 다음 시점으로 전달된다

실제로 은닉 상태 크기가 3인 RNN에 "나는 오늘 밥을 먹었다"를 넣고, 시점마다 은닉 상태를 꺼내 봤습니다(실습 2). 단어를 하나 읽을 때마다 기억 3칸이 갱신됩니다. 또 같은 단어 "먹었다"로 끝나도 "나는 밥을 먹었다"와 "오늘 먹었다"의 마지막 은닉 상태는 서로 다릅니다. **앞 문맥이 기억에 남아 있기 때문**입니다.

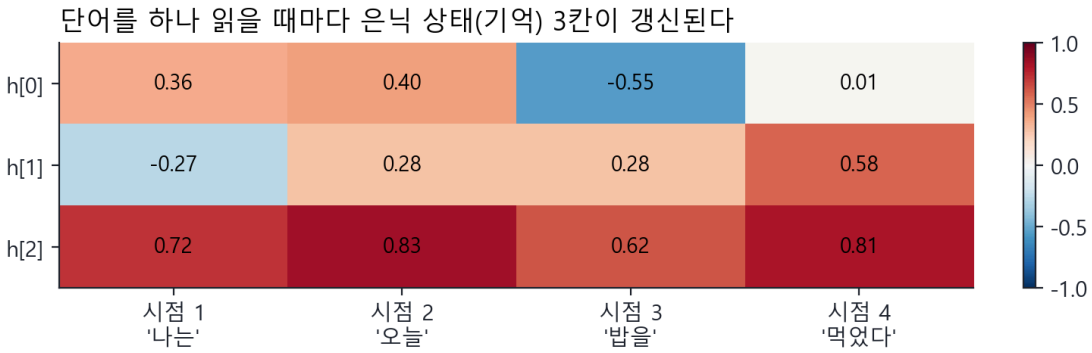


그림 2 "나는 오늘 밥을 먹었다"를 읽으며 갱신되는 은닉 상태(실습 2와 같은 값). 각 칸은 h_t의 한 원소

3) 입출력 구성, 깊은 RNN, 양방향 RNN

정의

- **입출력 구성**: 입력과 출력의 길이를 자유롭게 정할 수 있습니다. **일 대 다**(입력 하나 → 시퀀스), **다 대 일**(시퀀스 → 출력 하나), **다 대 다**(시퀀스 → 시퀀스)
- 깊은 RNN: 은닉층(RNN 층)을 여러 개 쌓은 것. 아래층의 시점별 출력이 위층의 입력이 됩니다. `num_layers=2`
- 양방향 RNN: 앞→뒤로 읽는 RNN과 뒤→앞으로 읽는 RNN을 함께 쓰고 두 은닉 상태를 이어 붙입니다. 뒤 단어가 단서가 될 때 유용합니다. `bidirectional=True`
- 파이토치 `nn.RNN(input_size, hidden_size, batch_first=True)`
  - `input_size` : 입력 x_t 벡터의 크기. 예: $x_t = [1, 2, 3]$이면 3
  - `hidden_size` : 은닉 상태 h_t 벡터의 크기
  - `batch_first=True` : 입력 모양이 (배치 수, 시점 수 = 단어 개수, 입력 크기). `False` (기본값)이면 (시점 수, 배치 수, 입력 크기)
  - 출력은 `output, h_n` 두 개입니다. `output` 은 (맨 위층의) 모든 시점의 은닉 상태, `h_n` 은 층마다·방향마다의 마지막 은닉 상태이며 모양은 (층 수 × 방향 수, 배치, 은닉 크기)입니다.
- 패딩: 한 배치 안의 문장들은 길이가 같아야 하나의 텐서로 묶을 수 있습니다. 그래서 가장 긴 문장에 맞춰 짧은 문장의 빈칸을 의미 없는 `<PAD>` 토큰으로 채웁니다(09-05).

예시 적용	일 대 다	다 대 일	다 대 다

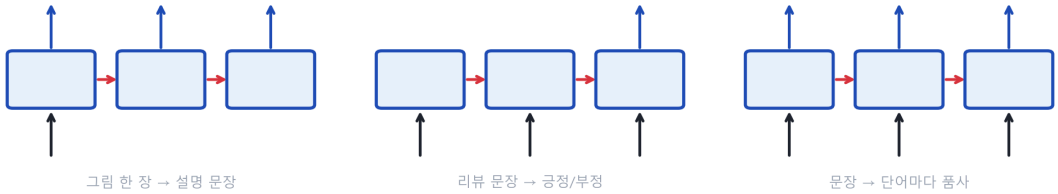


그림 3 RNN의 입출력 구성. 아래 화살표가 입력, 위 화살표가 출력이다

- 영화 리뷰 한 줄을 다 읽고 마지막에 긍정/부정을 하나 내면 **다 대 일**(13장 텍스트 분류)
- 문장의 단어마다 품사나 개체명을 붙이면 **다 대 다**(14장 시퀀스 레이블링)
- "나는 ___ 를 먹었다"의 빈칸을 채울 때는 뒤의 "먹었다"도 단서이므로 **양방향 RNN**이 유리합니다.

입력 모양 읽기. `batch_first=True`에서 입력이 (3, 5, 5)라면, 각 단어는 원소 5개짜리 벡터이고, 그런 단어 5개가 모여 한 문장을 이루며, 그런 문장이 3개 있다는 뜻입니다. 세 문장의 길이가 다르면 가장 긴 문장(4단어)에 맞춰 패딩합니다.

원래 문장	단어 수	패딩 후 (4단어)
I love AI	3	I love AI <PAD>
PyTorch is very fun	4	PyTorch is very fun
Hello	1	Hello <PAD> <PAD> <PAD>

깊은·양방향 RNN의 출력 모양. 단어 10개(입력 크기 5)짜리 문장 1개, 은닉 크기 8일 때(실습 1) 단계마다 데이터의 shape을 따라가 봅니다. 칸 하나가 숫자 하나입니다.

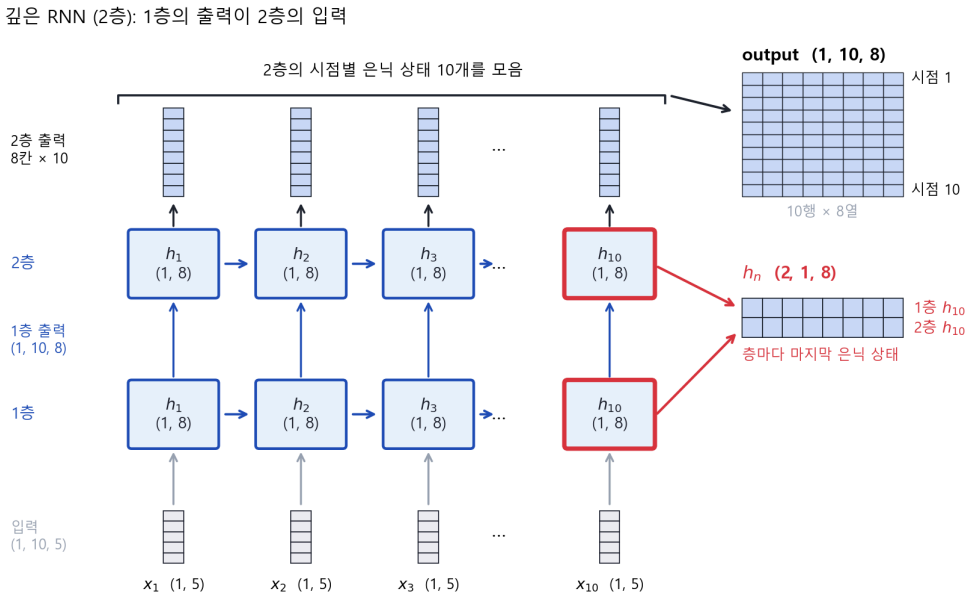


그림 4 깊은 RNN. 시점마다 x_t (1, 5)가 1층에서 h_t (1, 8)이 되고, 그것이 다시 2층의 입력이 된다. 2층의 h_t 10개를 모은 것이 output (1, 10, 8), 층마다의 마지막 상태(빨간 상자) 2개를 쌓은 것이 h_n (2, 1, 8)이다

양방향 RNN: 같은 입력을 정방향과 역방향이 각각 읽음

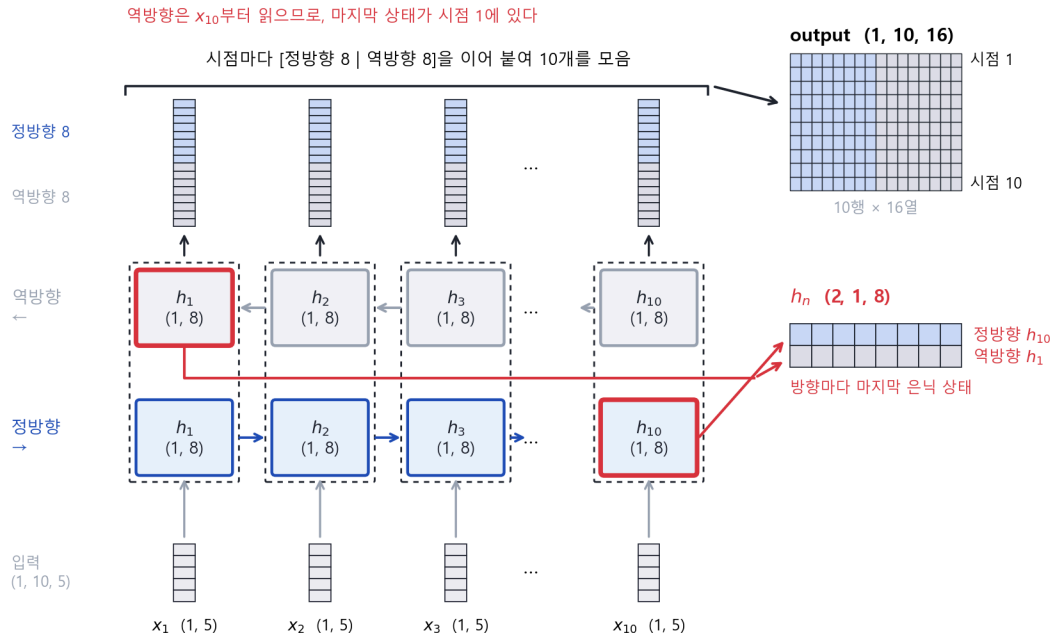


그림 5 양방향 RNN. 같은 x_t를 두 방향이 각각 읽어 (1, 8)씩 만들고, 이어 붙여 시점마다 (1, 16)이 된다. 이를 10개 모은 것이 output (1, 10, 16)이고, h_n (2, 1, 8)은 정방향의 시점 10과 역방향의 시점 1 상태다

구조	데이터 흐름	output	h_n
깊은 RNN (2층)	(1,10,5) → 1층 → (1,10,8) → 2층 → (1,10,8). 1층의 은닉 상태 10개가 2층의 입력	(1, 10, 8): 2층의 모든 시점 은닉 상태	(2, 1, 8): [1층 마지막, 2층 마지막]
양방향 RNN	정방향과 역방향이 같은 문장을 각각 읽음	(1, 10, 16): 시점마다 정방향 8 + 역방향 8을 이어 붙임	(2, 1, 8): [정방향 마지막 (시점 10), 역방향 마지막 (시점 1)]

비유

소설을 읽는 사람

소설을 읽을 때 우리는 페이지마다 처음부터 다시 읽지 않습니다. 지금까지의 줄거리를 머릿속에 요약해 두고, 새 문장을 읽을 때마다 그 요약을 고칩니다. 읽는 방식(습관)은 첫 페이지나 마지막 페이지나 같습니다.

지금 읽는 문장 = 현재 입력 x_t
머릿속 줄거리 요약 = 은닉 상태 h_t
늘 같은 읽는 습관 = 모든 시점이 공유하는 가중치

실습 1

nn.RNN의 입출력 모양과 내부 계산

```
import torch
import torch.nn as nn

torch.manual_seed(0)
# 단어 10개짜리 문장 1개, 각 단어는 숫자 5개로 표현되어 있다고 가정
inputs = torch.randn(1, 10, 5)            # (배치, 시점 수, 단어 하나의 크기)
```

```
cell = nn.RNN(input_size=5, hidden_size=8, batch_first=True)
outputs, h_last = cell(inputs)
print("모든 시점의 출력:", tuple(outputs.shape))   # 시점마다 은닉 상태(크기 8)가 하나씩
print("마지막 은닉 상태:", tuple(h_last.shape))

# RNN 안에서 일어나는 일을 직접 반복문으로: 이전 기억(h)과 현재 입력(x)을 섞는다
W_x, W_h, b = cell.weight_ih_l0, cell.weight_hh_l0, cell.bias_ih_l0 + cell.bias_hh_l0
h = torch.zeros(8)
for t in range(10):
    h = torch.tanh(W_x @ inputs[0, t] + W_h @ h + b)   # h_t = tanh(Wx·x_t + Wh·h_(t-1)
+ b)
print("직접 계산과 nn.RNN 결과가 같은가?", torch.allclose(h, h_last[0, 0], atol=1e-6))

deep = nn.RNN(5, 8, num_layers=2, batch_first=True)                # 층을 2개 쌓기
bi = nn.RNN(5, 8, batch_first=True, bidirectional=True)            # 앞뒤 양방향
print("깊은 RNN 마지막 은닉 상태:", tuple(deep(inputs)[1].shape))   # (층 수, 배치, 은닉 크
기)
print("양방향 RNN 출력:", tuple(bi(inputs)[0].shape))              # 은닉 크기가 2배(8+8)
```

실행 결과

```
모든 시점의 출력: (1, 10, 8)
마지막 은닉 상태: (1, 1, 8)
직접 계산과 nn.RNN 결과가 같은가? True
깊은 RNN 마지막 은닉 상태: (2, 1, 8)
양방향 RNN 출력: (1, 10, 16)
```

직접 해 보기 `hidden_size=8` 을 16으로 바꾸면 출력 모양의 어느 숫자가 바뀌나요? 양방향 RNN의 출력이 왜 16인지도 정의로 설명해 보세요.

실습 2 문장을 한 단어씩 읽으며 은닉 상태 따라가기

```
import torch
import torch.nn as nn

torch.manual_seed(0)
# 단어 4개짜리 단어 집합. 단어마다 원-핫 벡터로 표현
vocab = ["나는", "오늘", "밥을", "먹었다"]
onehot = torch.eye(len(vocab))

cell = nn.RNN(input_size=4, hidden_size=3, batch_first=True)   # 은닉 상태 크기 3

def read(sentence):
    x = torch.stack([onehot[vocab.index(w)] for w in sentence.split()]).unsqueeze(0)
    outputs, h_last = cell(x)                    # outputs: 시점마다의 은닉 상태
    return outputs[0], h_last

# 1) 한 단어씩 읽을 때마다 은닉 상태(기억)가 바뀐다
outputs, _ = read("나는 오늘 밥을 먹었다")
for t, (w, h) in enumerate(zip("나는 오늘 밥을 먹었다".split(), outputs)):
    print(f"시점 {t + 1} '{w}' 읽은 뒤 은닉 상태: {[round(v, 3) for v in h.tolist()]}")

# 2) 같은 단어 '먹었다'라도 앞 문맥이 다르면 은닉 상태가 다르다
_, h_a = read("나는 밥을 먹었다")
_, h_b = read("오늘 먹었다")
print("'나는 밥을 먹었다' 끝 상태:", [round(v, 3) for v in h_a[0, 0].tolist()])
print("'오늘 먹었다'      끝 상태:", [round(v, 3) for v in h_b[0, 0].tolist()])

# 3) 같은 RNN 하나로 길이가 다른 문장을 모두 처리한다 (가중치 공유)
for s in ["먹었다", "나는 먹었다", "나는 오늘 밥을 먹었다"]:
    print(f"{s!r:22s} → 출력 모양 {tuple(read(s)[0].shape)}")
```

실행 결과

```
시점 1 '나는' 읽은 뒤 은닉 상태: [0.364, -0.273, 0.718]
시점 2 '오늘' 읽은 뒤 은닉 상태: [0.404, 0.283, 0.829]
시점 3 '밥을' 읽은 뒤 은닉 상태: [-0.549, 0.283, 0.62]
시점 4 '먹었다' 읽은 뒤 은닉 상태: [0.009, 0.583, 0.806]
'나는 밥을 먹었다' 끝 상태: [-0.063, 0.58, 0.797]
```

```
'오늘 먹었다'      끝 상태: [-0.536, 0.613, 0.635]
'먹었다'                  → 출력 모양 (1, 3)
'나는 먹었다'              → 출력 모양 (2, 3)
'나는 오늘 밥을 먹었다'       → 출력 모양 (4, 3)
```

직접 해 보기 "밥을 나는 먹었다"처럼 순서만 바꾼 문장의 마지막 은닉 상태를 "나는 밥을 먹었다"와 비교해 보세요. 같은 단어인데 왜 다를까요?

확인 문제

07-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 피드 포워드 신경망은 이전 입력에 대한 기억이 없다. (O / X) 이름 _____

2 RNN은 시점마다 서로 다른 가중치를 쓴다. (O / X) 이름 _____

3 $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$에서 h_{t-1}의 뜻은? 이름 _____

① 현재 입력 ② 직전 시점의 은닉 상태(기억) ③ 정답

4 리뷰 문장을 다 읽고 긍정/부정을 하나 내는 구성은? 이름 _____

① 일 대 다 ② 다 대 일 ③ 다 대 다

5 시퀀스에서 몇 번째 원소인지를 나타내는 것을 _____이라고 한다. 이름 _____

6 실습 2에서 같은 "먹었다"로 끝나도 마지막 은닉 상태가 달랐던 이유는 앞 _____이 기억에 남았기 때문이다. 이름 _____

7 은닉 상태를 정의하고, "나는 오늘 밥을 먹었다"를 읽는 과정에 적용해 설명해 보세요. 이름 _____

원문: 07-01 순환 신경망 — https://wikidocs.net/60690

07-02 LSTM과 GRU

핵심 한 줄

바닐라 RNN은 시퀀스가 길어지면 앞쪽 정보를 잊는다(장기 의존성 문제). LSTM은 셀 상태와 세 게이트로 무엇을 지우고, 저장하고, 내보낼지 골라 긴 시퀀스에서도 기억을 지킨다.

왜 배우나

실제 문장은 깁니다. "내가 어제 파리에서 … (긴 이야기) … 그래서 나는 ___ 를 할 줄 안다"의 빈칸(프랑스어)을 맞히려면 맨 앞의 "파리"를 끝까지 기억해야 합니다. 바닐라 RNN은 이것을 잘 못하고, LSTM과 GRU는 이 문제를 풀어 트랜스포머 이전까지 자연어 처리의 기본 모델이었습니다.

1) 바닐라 RNN의 한계: 장기 의존성 문제

정의

지금까지의 가장 단순한 RNN을 바닐라 RNN이라고 합니다. 역전파 때 기울기는 시점을 거슬러 올라가며 매번 W_h와 tanh의 기울기(1 이하)를 곱하므로, 시퀀스가 길면 앞쪽 시점에 도달하는 기울기가 거의 0이 됩니다(06-08의 기울기 소실이 시간 방향으로 일어나는 것). 그 결과 앞쪽 입력의 정보가 뒤쪽 출력에 거의 영향을 주지 못합니다. 이를 장기 의존성 문제(the problem of long-term dependencies)라고 합니다.

부연설명

바닐라 RNN은 지금까지의 내용을 고정 크기 벡터 h_t 하나에 요약해 나가고, 모든 시점이 같은 W_x, W_h를 씁니다. 이 구조 때문에 두 방향에서 문제가 생깁니다.

- **순전파: 앞 정보가 흐려진다.** 매 시점 $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$로 새로 계산되므로, 이전 정보는 W_h와 tanh를 거칠 때마다 조금씩 줄고 그 위에 새 단어 정보가 덧씌워집니다. 그래서 첫 단어의 정보는 문장 끝의 h_T에 거의 남지 않습니다.
- **역전파: 고치라는 신호도 약하다.** 오차는 끝에서부터 거슬러 오며 시점마다 1보다 작은 값이 곱해져 0.5 → 0.25 → 0.125…처럼 급격히 작아집니다. W_x, W_h는 모든 시점이 공유하므로 각 시점의 기울기를 모두 더해 한 번에 갱신하는데, 가까운 시점의 큰 기울기에 앞쪽 시점의 작은 기울기가 묻혀 버립니다. 결국 가중치는 가까운 단어끼리의 관계만 학습하고, "앞 내용을 끝까지 보존하는 법"은 배울 기회를 놓칩니다.

즉 순전파에서 앞 정보가 흐려지고, 역전파에서도 그것을 고치라는 신호가 약해서 끝내 고쳐지지 않는 구조입니다.

예시 적용

입력 50개를 읽은 뒤, 각 위치의 입력이 마지막 출력에 미치는 영향(기울기 크기)을 바닐라 RNN과 LSTM에서 재 봤습니다(실습 5). 마지막 입력의 영향을 1이라고 할 때, 맨 앞 입력의 영향은 바닐라 RNN에서 약 2.4×10^{-11}으로 사실상 0이고, LSTM에서는 약 0.23으로 남아 있습니다.

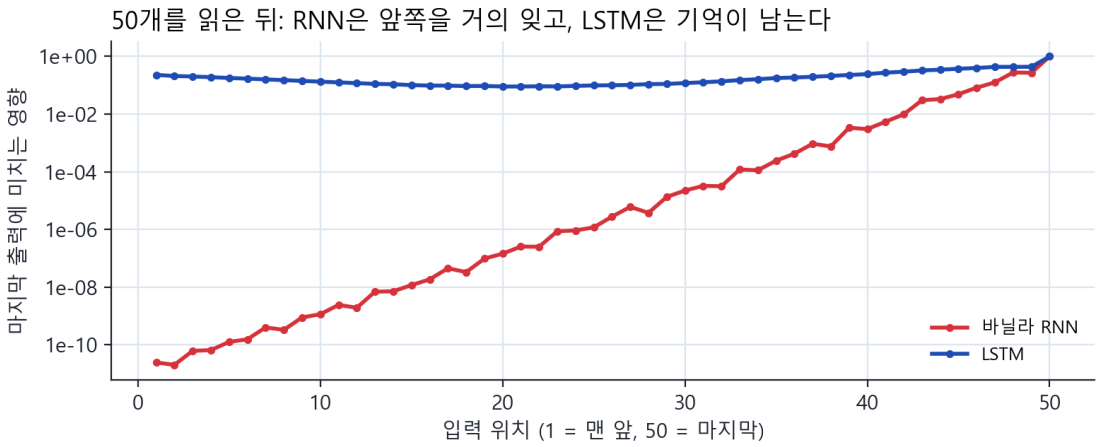


그림 6 50개를 읽은 뒤 위치별로 남은 영향(로그 눈금). 바닐라 RNN은 앞쪽으로 갈수록 영향이 급격히 사라지고, LSTM은 거의 일정하게 유지된다

2) LSTM: 셀 상태와 세 개의 게이트

정의

LSTM(Long Short-Term Memory)은 은닉 상태 h_t(단기 기억) 외에 셀 상태 C_t(장기 기억)를 따로 두고, 0~1 사이 값으로 정보의 양을 조절하는 게이트 세 개로 기억을 관리합니다. σ는 시그모이드, $\odot$는 같은 자리끼리의 곱입니다.

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i), \quad g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$$
$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$$
$$C_t = f_t \odot C_{t-1} + i_t \odot g_t$$
$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(C_t)$$

- **새 정보 후보** g_t: 현재 입력 x_t와 직전 기억 h_{t-1}로 만든 "저장할 수도 있는 새 내용". tanh라서 -1~1 사이 값이며, 게이트가 아니라 **내용**입니다.
- **입력 게이트** i_t: 새 정보 후보 g_t 중 얼마나 저장할지 (0~1). 셀 상태에 실제로 더해지는 양은 $i_t \odot g_t$ 입니다.
- **삭제 게이트** f_t: 이전 장기 기억 C_{t-1} 중 얼마나 남길지 (0이면 지움, 1이면 유지)
- **셀 상태** C_t: 남긴 옛 기억 + 저장한 새 정보. 곱셈과 덧셈만 거쳐 흐르므로 기울기가 잘 사라지지 않습니다.
- **출력 게이트** o_t: 장기 기억 중 지금 시점에 꺼내 쓸 것을 골라 은닉 상태 h_t로 내보냄

참고만

왜 셀 상태에서는 기울기가 잘 사라지지 않을까? 역전파 때 한 시점을 거슬러 갈 때마다 곱해지는 값을 비교해 봅니다.

$$\text{바닐라 RNN: } \frac{\partial h_t}{\partial h_{t-1}} = \tanh'(\cdot)\, W_h \qquad \text{LSTM 셀 상태: } \frac{\partial C_t}{\partial C_{t-1}} = f_t$$

바닐라 RNN은 h_{t-1}이 h_t가 될 때마다 W_h를 곱하고 tanh를 통과하므로, 거슬러 갈 때도 매번 1보다 작아지기 쉬운 값이 곱해집니다. 셀 상태는 $C_t = f_t \odot C_{t-1} + i_t \odot g_t$라서 옛 기억에는 f_t만 곱해지고(가중치 행렬도 tanh도 없음), 새 정보는 **덧셈**으로 붙습니다. 덧셈은 기울기를 줄이지 않고 그대로 통과시키므로, 거슬러 갈 때 곱해지는 값은 f_t 하나뿐입니다. 모델이 "기억할 때"라고 판단해 $f_t \approx 0.95$로 두면 50시점을 거슬러도 $0.95^{50} \approx 0.08$이 남습니다. 매번 0.5씩 줄어든다면 $0.5^{50} \approx 10^{-15}$으로 사라집니다.

예시 적용

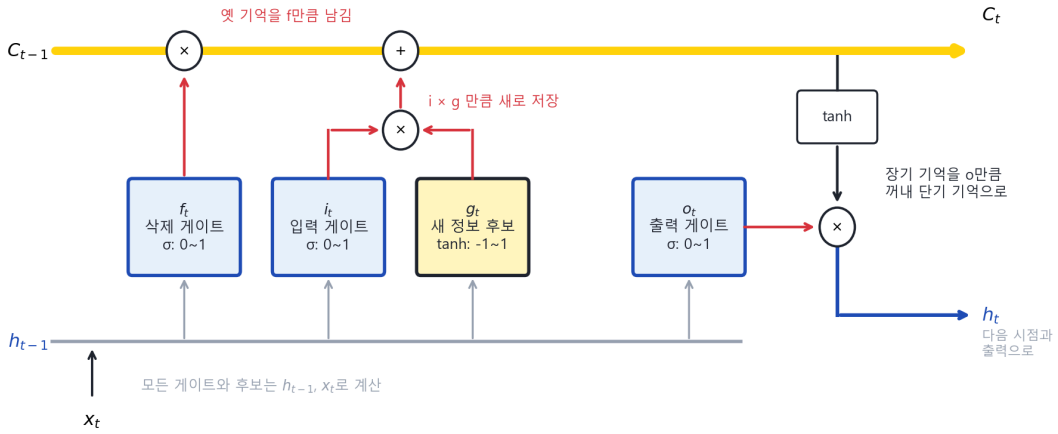


그림 7 LSTM 셀 하나. 모든 게이트와 후보는 h_{t-1}, x_t로 계산된다. 셀 상태(노랑)에는 f가 곱해지고 $i \times g$가 더해지며, 그렇게 만든 C_t를 tanh와 o를 거쳐 꺼낸 것이 h_t다

"내가 어제 **파리**에서 … 그래서 나는 **프랑스어**를 할 줄 안다"를 읽는 LSTM을 생각해 봅시다. "파리"를 읽을 때 입력 게이트가 열려 장소 정보를 셀 상태에 저장하고, 중간의 관계없는 단어들을 읽는 동안 삭제 게이트가 1에 가까워 그 정보를 유지합니다. 빈칸 직전에 출력 게이트가 그 기억을 꺼내 "프랑스어"를 맞히는 데 씁니다. 위 실험(실습 5)에서 삭제 게이트 편향을 1로 두어 "처음에는 잘 지우지 않게" 시작한 것도 같은 원리입니다.

참고만

왜 삭제 게이트 편향을 1로 둘까? 학습 전에는 가중치가 무작위 값이라, 모델이 아직 어떤 단어가 중요한지 데이터에서 배우지 못한 상태입니다. 이때 f가 $\sigma(0) = 0.5$ 근처에서 시작하면 앞 단어의 기억이 매 시점 절반씩 지워져서, 중요할 수도 있는 초반 단어가 잘못 지워집니다. 기억이 지워지면 역전파 신호도 앞까지 가지 못해 "이 단어는 남겨야 한다"를 배울 기회도 사라집니다. 편향을 1로 두면 f가 0.5보다 큰 값에서 시작해 "일단은 지우지 않는" 쪽으로 출발하고, 무엇을 지울지는 학습하면서 정합니다. (실습 코드는 파이토치 LSTM의 두 편향 `bias_ih`, `bias_hh`를 모두 1로 두므로 합이 2이고, 학습 전 $f \approx \sigma(2) \approx 0.88$에서 시작합니다.)

숫자로 따라가기. 기억이 1칸뿐인 LSTM이 세 시점을 읽는다고 하고, 정의의 식에 그대로 값을 넣어 봅니다 (설명용 예시 값).

	f 삭제 게이트	i 입력 게이트	g 새 정보 후보	f·C(t-1) 남긴 옛 기억	i·g 저장한 새 정보	C(t) 새 장기 기억	o 출력 게이트	h(t) 꺼낸 기억	
시점 1 '파리에서'	0.10	0.90	0.80	0.00	0.72	**0.72**	0.20	**0.12**	장소 정보 저장
시점 2 '날씨가 좋았다'	0.95	0.05	-0.30	0.68	-0.01	**0.67**	0.10	**0.06**	관계없는 단어: 기억 유지
시점 3 '나는 __ 를'	0.95	0.10	0.10	0.64	0.01	**0.65**	0.95	**0.54**	기억을 꺼내 씀

설명용 예시 값(기억 1칸). C(t) = f·C(t-1) + i·g, h(t) = o·tanh(C(t))

그림 8 시점마다 f, i, g로 새 장기 기억 $C_t = f \cdot C_{t-1} + i \cdot g$를 만들고, o로 꺼내 $h_t = o \cdot \tanh(C_t)$를 낸다

1. **시점 1 "파리에서"**: 삭제 게이트 $f = 0.1$이므로 옛 기억은 10%만 남깁니다(문장 처음이라 $C_0 = 0$이기도 함). 새 정보 후보 $g = 0.8$을 입력 게이트 $i = 0.9$로 많이 저장 → $C_1 = 0.72$. 아직 쓸 때가 아니라 출력 게이트 $o = 0.2$ → $h_1 = 0.12$로 조금만 내보냄
2. **시점 2 "날씨가 좋았다"**: 관계없는 내용이라 $i = 0.05$로 거의 저장하지 않고, $f = 0.95$로 옛 기억을 유지 → $C_2 = 0.67$. 장소 정보가 그대로 남음

3. **시점 3 "나는 ___ 를"**: 빈칸을 채울 때라 출력 게이트 $o = 0.95$로 기억을 꺼냄 → $h_3 = 0.54$. 이 값으로 "프랑스어"를 예측

바닐라 RNN이었다면 시점 2에서 h가 새 단어로 덧씌워져 장소 정보가 흐려졌을 것입니다. 실제 LSTM 셀 하나에서 한 시점의 i, f, g, o, C, h를 계산해 파이토치 결과와 같은지 확인하는 것이 실습 4입니다.

3) GRU: 게이트를 두 개로 줄인 LSTM

정의

GRU(Gated Recurrent Unit)는 LSTM의 장기 의존성 해결 능력은 유지하면서 구조를 단순하게 만든 모델입니다. 셀 상태 없이 은닉 상태 하나만 쓰고, **업데이트 게이트** z_t와 **리셋 게이트** r_t 두 개만 둡니다.

참고만

셀 상태가 없는데 장기 기억은 어떻게 유지할까? 아래 식의 마지막 줄 $h_t = (1 - z_t) \odot g_t + z_t \odot h_{t-1}$에서 $z_t \approx 1$이면 h_{t-1}이 가중치 행렬도 tanh도 거치지 않고 그대로 h_t로 넘어갑니다. LSTM 셀 상태처럼 "곱셈 하나와 덧셈"만 있는 길이 은닉 상태 안에 생기는 것이고, 역전파 때 한 시점마다 곱해지는 값도 z_t입니다. 즉 GRU는 LSTM의 C(장기 기억)와 h(단기 기억)를 h 하나로 합쳐 두 역할을 함께 맡기고, 출력 게이트가 없어 기억 전체를 그대로 내보냅니다.

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r), \quad z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z)$$
$$g_t = \tanh\big(W_{hg}(r_t \odot h_{t-1}) + W_{xg}x_t + b_g\big)$$
$$h_t = (1 - z_t) \odot g_t + z_t \odot h_{t-1}$$

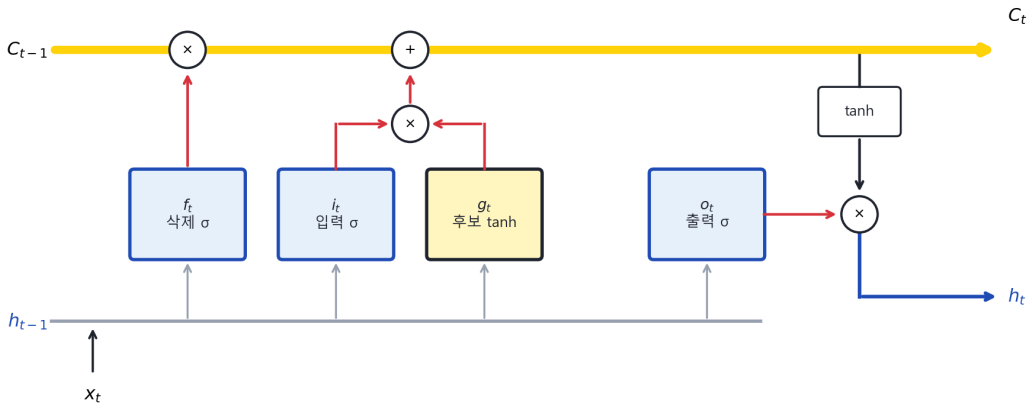
- **리셋 게이트** r_t (0~1): 새 정보 후보를 만들 때 직전 기억 h_{t-1}을 **얼마나 반영할지**. 0에 가까우면 옛 기억을 무시하고 현재 입력 위주로 후보를 만듭니다.
- **업데이트 게이트** z_t (0~1): 새 은닉 상태에서 **옛 기억과 새 정보의 비율**. LSTM의 삭제 게이트와 입력 게이트를 하나로 합친 역할입니다(옛 기억을 z만큼 남기면 새 정보는 $1 - z$만큼 들어감).
- **새 정보 후보** g_t (-1~1): 리셋 게이트로 걸러 낸 옛 기억 $r_t \odot h_{t-1}$과 현재 입력으로 만든 새 내용
- **은닉 상태** h_t: z_t가 1에 가까우면 이전 기억을 그대로 유지하고, 0에 가까우면 새 정보 g_t로 바꿉니다. 셀 상태가 따로 없고 이것이 유일한 기억입니다.

성능은 LSTM과 비슷하고 학습할 가중치가 적습니다.

예시 적용

입력 크기 5, 은닉 상태 크기 8일 때 학습할 숫자는 RNN 120개, LSTM 480개(게이트·후보 4세트), GRU 360개(3세트)입니다(실습 3). 데이터가 적으면 가벼운 GRU, 많으면 LSTM이 조금 낫다는 경험칙이 있지만, 보통은 둘 다 시도해 봅니다.

LSTM: 기억 2개(C, h), 게이트 3개(f, i, o) + 후보 g



GRU: 기억 1개(h), 게이트 2개(r, z) + 후보 g

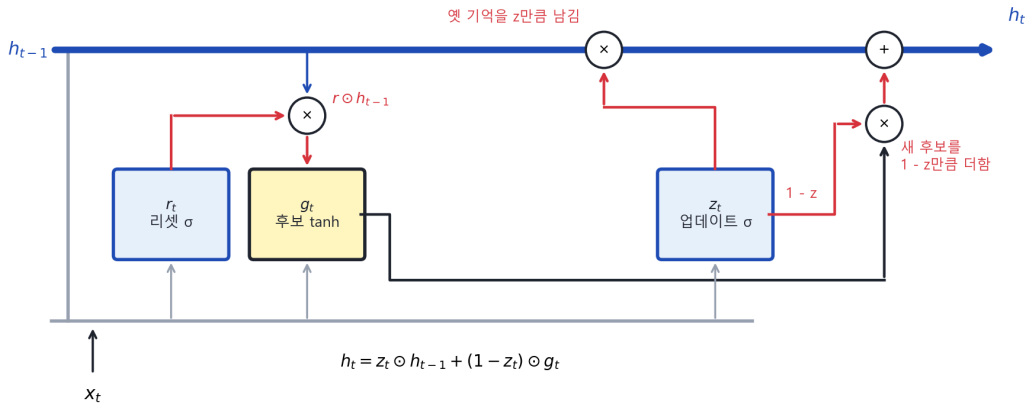


그림 9 위: LSTM은 셀 상태 C에 f를 곱하고 i×g를 더한 뒤, tanh와 o를 거쳐 h를 꺼낸다. 아래: GRU는 r로 걸러 낸 옛 기억 $r \odot h_{t-1}$로 후보 g를 만들고, h 선 위에서 옛 기억을 z만큼 남기고 g를 1-z만큼 더한다

	LSTM	GRU
기억	셀 상태 C_t(장기) + 은닉 상태 h_t(단기), 2개	은닉 상태 h_t 1개
게이트	삭제 f, 입력 i, 출력 o: 3개	리셋 r, 업데이트 z: 2개
옛 기억 남기기	삭제 게이트 f	업데이트 게이트 z
새 정보 넣기	입력 게이트 i (f와 따로 정함)	$1 - z$ (남긴 만큼 덜 넣음)
기억 꺼내기	출력 게이트 o로 C에서 골라 h로	따로 없음. h가 곧 출력
파라미터 수 (입력 5, 은닉 8)	480개 (4세트)	360개 (3세트)
파이토치 출력	`out, (h_n, c_n)`	`out, h_n`

비유

수업 노트 정리 LSTM

매 수업마다 노트를 새로 쓰는 대신 한 권의 정리 노트를 계속 들고 다닙니다. 시험 범위에서 빠진 내용은 지우고(삭제 게이트), 오늘 배운 것 중 중요한 것만 옮겨 적고(입력 게이트), 오늘 숙제에 필요한 부분만 펼쳐 봅니다(출력 게이트). 정리 노트가 있으니 학기 초 내용도 잊지 않습니다.

정리 노트	= 셀 상태 C_t (장기 기억)
지금 펼친 페이지	= 은닉 상태 h_t (단기 기억)
지우기, 옮겨 적기, 펼쳐 보기	= 삭제, 입력, 출력 게이트

실습 3

RNN, LSTM, GRU 비교

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
inputs = torch.randn(1, 10, 5)              # (배치 1, 시점 10, 입력 크기 5)

rnn = nn.RNN(5, 8, batch_first=True)
lstm = nn.LSTM(5, 8, batch_first=True)
gru = nn.GRU(5, 8, batch_first=True)

out, (h, c) = lstm(inputs)                  # LSTM은 기억이 두 개: h(단기)와 c(장기, 셀 상태)
print("LSTM 출력:", tuple(out.shape), " h:", tuple(h.shape), " c:", tuple(c.shape))
out, h = gru(inputs)                        # GRU는 다시 기억 하나(h)로 단순화
print("GRU  출력:", tuple(out.shape), " h:", tuple(h.shape))

count = lambda m: sum(p.numel() for p in m.parameters())
print("학습할 숫자(파라미터) 개수")
print("  RNN :", count(rnn))
print("  LSTM:", count(lstm), " (게이트 4개 → RNN의 4배)")
print("  GRU :", count(gru), " (게이트 3개 → RNN의 3배)")
```

실행 결과

```
LSTM 출력: (1, 10, 8)  h: (1, 1, 8)  c: (1, 1, 8)
GRU  출력: (1, 10, 8)  h: (1, 1, 8)
학습할 숫자(파라미터) 개수
  RNN : 120
  LSTM: 480  (게이트 4개 → RNN의 4배)
  GRU : 360  (게이트 3개 → RNN의 3배)
```

직접 해 보기 LSTM의 출력은 왜 `out, (h, c)` 두 묶음이고, GRU는 왜 `out, h` 일까요? 정의의 h_t, C_t와 연결해 설명해 보세요.

실습 4

LSTM 한 시점을 식 그대로 계산하기: i, f, g, o, C, h

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
# 입력 크기 2, 은닉 크기 3인 LSTM 셀 하나로 "한 시점"의 계산을 식 그대로 따라가 본다
cell = nn.LSTMCell(input_size=2, hidden_size=3)
x_t = torch.tensor([1.0, -0.5])                 # 지금 들어온 단어 벡터
h_prev = torch.tensor([0.2, -0.1, 0.4])         # 직전 단기 기억 h_(t-1)
C_prev = torch.tensor([0.9, -0.3, 0.5])         # 직전 장기 기억 C_(t-1)

# 파이토치는 네 덩어리(i, f, g, o)의 가중치를 한 행렬에 이어 붙여 둔다
z = cell.weight_ih @ x_t + cell.bias_ih + cell.weight_hh @ h_prev + cell.bias_hh
z_i, z_f, z_g, z_o = z.chunk(4)
i = torch.sigmoid(z_i)          # 입력 게이트: 새 정보를 얼마나 저장할까 (0~1)
```

```
f = torch.sigmoid(z_f)          # 삭제 게이트: 옛 기억을 얼마나 남길까 (0~1)
g = torch.tanh(z_g)             # 새 정보 후보: 무엇을 저장할까 (-1~1)
o = torch.sigmoid(z_o)          # 출력 게이트: 기억을 얼마나 꺼내 쓸까 (0~1)
C = f * C_prev + i * g          # 새 장기 기억 = 남긴 옛 기억 + 저장한 새 정보
h = o * torch.tanh(C)           # 새 단기 기억 = 꺼내 쓴 장기 기억

show = lambda name, v: print(f"{name:22s} {[round(x, 3) for x in v.tolist()]}")
show("i  입력 게이트", i)
show("f  삭제 게이트", f)
show("g  새 정보 후보", g)
show("o  출력 게이트", o)
show("f*C_prev 남긴 옛 기억", f * C_prev)
show("i*g  저장한 새 정보", i * g)
show("C  새 장기 기억", C)
show("h  새 단기 기억", h)

h_torch, C_torch = cell(x_t.unsqueeze(0), (h_prev.unsqueeze(0), C_prev.unsqueeze(0)))
print("직접 계산 = nn.LSTMCell 결과?", torch.allclose(h, h_torch[0], atol=1e-6), torch.a
llclose(C, C_torch[0], atol=1e-6))
```

실행 결과

```
i  입력 게이트              [0.626, 0.579, 0.405]
f  삭제 게이트              [0.553, 0.572, 0.319]
g  새 정보 후보             [-0.362, -0.411, -0.026]
o  출력 게이트              [0.716, 0.53, 0.216]
f*C_prev 남긴 옛 기억       [0.498, -0.172, 0.16]
i*g  저장한 새 정보          [-0.226, -0.238, -0.011]
C  새 장기 기억             [0.272, -0.41, 0.149]
h  새 단기 기억             [0.19, -0.206, 0.032]
직접 계산 = nn.LSTMCell 결과? True True
```

직접 해 보기 f를 모두 1, i를 모두 0으로 강제로 바꾸면 C는 어떻게 될까요? 정의의 $C_t = f \odot C_{t-1} + i \odot g_t$로 예측해 보세요.

실습 5 앞쪽 입력이 끝까지 영향을 주는가: RNN vs LSTM

```
import torch
import torch.nn as nn

# 50개짜리 수열을 읽은 뒤, "몇 번째 입력이 마지막 출력에 얼마나 영향을 주는가"를 기울기로 잰다.
# 영향이 0에 가까우면 그 입력은 마지막 시점까지 기억되지 못한 것이다.
def influence(kind, length=50, hidden=32, seeds=5):
    total = torch.zeros(length)
    for s in range(seeds):
        torch.manual_seed(s)
        model = (nn.RNN if kind == "RNN" else nn.LSTM)(1, hidden, batch_first=True)
        if kind == "LSTM":                          # 흔히 쓰는 설정: 삭제 게이트 편향을 1
로 (처음엔 잘 안 지우게)
            for name, p in model.named_parameters():
                if "bias" in name:
                    p.data[hidden:2 * hidden] = 1.0
        x = torch.randn(16, length, 1, requires_grad=True)
        out, _ = model(x)
        out[:, -1].sum().backward()                 # 마지막 시점의 출력에서 거꾸로
        total += x.grad.abs().mean(0)[:, 0]         # 위치별 영향의 크기
    return total / total[-1]                        # 마지막 입력의 영향을 1로 맞춰 비교

rnn, lstm = influence("RNN"), influence("LSTM")
print("입력 위치 | RNN에서의 영향 | LSTM에서의 영향  (마지막 입력 = 1)")
for t in [1, 11, 21, 31, 41, 50]:
    print(f"   {t:2d}     |   {rnn[t - 1].item():.1e}      |   {lstm[t - 1].item():.1
e}")
```

실행 결과

```
입력 위치 | RNN에서의 영향 | LSTM에서의 영향  (마지막 입력 = 1)
    1     |   2.4e-11      |   2.3e-01
   11     |   2.4e-09      |   1.3e-01
   21     |   2.5e-07      |   9.1e-02
```

```
31    |   3.2e-05     |   1.3e-01
41    |   5.4e-03     |   2.7e-01
50    |   1.0e+00     |   1.0e+00
```

직접 해 보기 `length=50`을 10으로 줄이면 바닐라 RNN의 맨 앞 영향은 얼마나 커지나요? 짧은 문장에서는 바닐라 RNN도 괜찮은 이유를 설명해 보세요.

확인 문제

07-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 바닐라 RNN은 시퀀스가 길어지면 앞쪽 정보를 잘 잊는다. (O / X) 이름 _____

2 GRU는 LSTM보다 게이트가 더 많다. (O / X) 이름 _____

3 LSTM에서 장기 기억을 담는 것은? 이름 _____

① 셀 상태 C_t ② 입력 x_t ③ 출력층

4 $C_t = f_t \odot C_{t-1} + i_t \odot g_t$에서 f_t가 0에 가까우면? 이름 _____

① 이전 기억을 지운다 ② 이전 기억을 그대로 유지한다 ③ 출력이 0이 된다

5 앞부분 정보가 뒤까지 전달되지 못하는 문제를 _____ 문제라고 한다. 이름 _____

6 실습 5에서 50개를 읽은 뒤 맨 앞 입력의 영향은 LSTM에서 약 _____였다. 이름 _____

7 LSTM의 세 게이트를 정의하고, "파리 … 프랑스어" 문장에 적용해 설명해 보세요. 이름 _____

원문: 07-02 LSTM과 GRU — https://wikidocs.net/60762

07-03 문자 단위 RNN(Char RNN)

핵심 한 줄

문자를 원-핫 벡터로 바꿔 시점마다 하나씩 넣고, 다 대 다 RNN이 시점마다 "다음 글자"의 확률을 내도록 크로스 엔트로피로 학습시킨다.

왜 배우나

07-01, 07-02에서는 RNN의 구조를 살펴봤습니다. 이번에는 RNN을 **학습**시켜 실제 문제를 풉니다. 과제는 "지금까지 본 글자로 다음 글자 맞히기"이고, 이것을 단어 단위로, 아주 큰 모델로 키운 것이 GPT 같은 언어 모델입니다(10장, 21장).

1) 문자 단위 RNN의 정의

정의

- 문자 단위 RNN(Char RNN)은 입출력의 단위가 단어가 아니라 **문자**인 RNN입니다.
- **문자 집합**: 데이터에 나온 문자의 목록. 각 문자에 번호를 붙이고 **원-핫 벡터**(05장)로 바꿔 입력합니다.
- **학습 데이터**: 입력 시퀀스와, 그것을 한 칸씩 밀린 정답 시퀀스. 시점 t의 정답은 $t+1$번째 문자입니다.
- **구성**: 다 대 다. 시점마다 은닉 상태를 `nn.Linear` 로 문자 집합 크기만큼의 점수로 바꾸고, 크로스 엔트로피로 학습합니다(05장 소프트맥스 회귀를 시점마다 하는 것).

시점 하나에서 벡터가 바뀌는 과정 (문자 집합 크기 V, 은닉 크기 H)

예측(추론 시) : C=예측한 문자(Character)

$$x_t \xrightarrow{\tanh(W_x x_t + W_h h_{t-1} + b)} h_t \xrightarrow{W_y h_t + b_y} s_t \xrightarrow{\text{softmax}} p_t \xrightarrow{\arg\max} \hat{c}_{t+1}$$

원-핫 (V) 　 (H) 　 점수 (V) 　 확률 (V)

원-핫 입력은 RNN 셀을 지나 은닉 상태가 되고, 선형층을 지나 문자마다 점수가 되며, 소프트맥스로 합이 1인 확률이 되어 가장 큰 문자를 다음 글자로 고릅니다. 학습할 때는 이 확률과 정답 문자로 크로스 엔트로피를 계산합니다.

학습 단계에서는 정답과 확률을 크로스 엔트로피에 넣어 손실 계산하고, 손실 적어지게 학습

예시 적용

입력 "apple", 정답 "pple!"이면 문자 집합은 ['!', 'a', 'e', 'l', 'p'] 5종류이고, 입력 텐서는 (문장 1개, 글자 5개, 원-핫 크기 5) = (1, 5, 5)입니다. a 다음은 p, p 다음은 p, … e 다음은 문장 끝 !입니다.

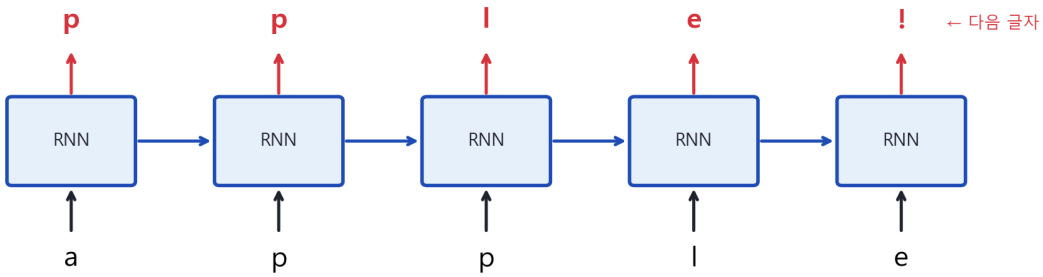


그림 10 Char RNN의 학습 목표. 시점마다 지금까지 읽은 글자로 다음 글자(빨강)를 맞힌다

처음에는 "lllll"처럼 엉뚱하게 예측하지만, 학습 5번 만에 "pple!"을 맞히기 시작합니다. 같은 "p"를 입력받아도 첫 번째 p 다음에는 p, 두 번째 p 다음에는 l을 내놓습니다. 은닉 상태에 "지금 몇 번째 p인지"라는 앞 문맥이 담겨 있기 때문입니다.



그림 11 실습과 같은 설정의 학습 곡선. 손실이 빠르게 0에 가까워진다

벡터로 따라가기. 학습된 모델에 "apple"을 넣고 시점마다 위 정의의 네 벡터를 꺼내 봤습니다(실습 7). 각 행이 시점 하나이고, 왼쪽에서 오른쪽으로 원-핫 → 은닉 상태 → 점수 → 확률 순서입니다.

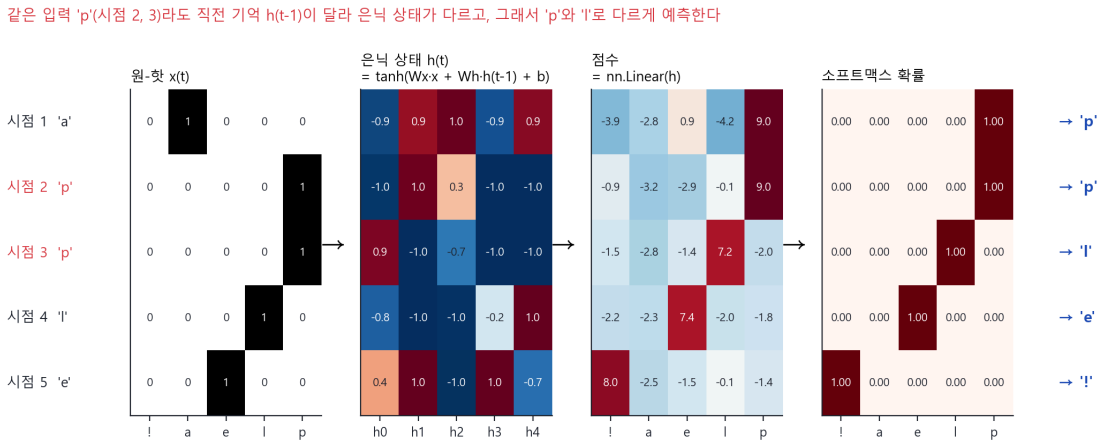


그림 12 시점마다 원-핫 입력이 은닉 상태, 점수, 소프트맥스 확률을 거쳐 다음 글자가 된다. 시점 2와 3은 입력이 같은 'p'지만 직전 기억이 달라 'p'와 'l'로 다르게 예측한다

시점 3을 예로 들면, 입력 'p'의 원-핫 [0, 0, 0, 0, 1]이 직전 기억과 섞여 은닉 상태 [0.92, -1.00, -0.70, -1.00, -1.00]이 되고, 선형층을 지나 점수에서 'l'이 7.24로 가장 크며, 소프트맥스 후 'l'의 확률이 1.00이 되어 다음 글자로 'l'을 고릅니다. 같은 'p'를 넣은 시점 2에서는 은닉 상태가 [-0.97, 0.97, 0.30, -1.00, -1.00]으로 달라 'p'의 점수(8.98)가 가장 큽니다.

교재의 두 번째 실습은 같은 방법을 긴 영어 문장("if you want to build a ship, …")으로 해 봅니다. 문장을 10글자씩 잘라 수많은 학습 예시를 만들고, RNN을 2층으로 쌓아 학습하면 문장 전체를 거의 그대로 이어 씁니다. 코랩에서 교재 코드를 그대로 돌려 보세요.

비유

휴대폰 키보드의 다음 글자 추천

키보드가 "안녕하"까지 치면 "세요"를 추천하는 것처럼, 많이 본 글자 순서를 기억해 두었다가 다음 글자를 예측합니다. 처음에는 엉뚱한 추천을 하지만, 틀린 만큼 고치기를 반복하면 규칙을 익힙니다.

지금까지 친 글자	= 입력 시퀀스 + 은닉 상태
추천 글자	= 시점마다 가장 큰 점수의 문자
추천이 맞았는지 확인	= 크로스 엔트로피

실습 6 apple로 다음 글자 맞히기

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
input_str, label_str = "apple", "pple!"      # 한 글자씩 밀린 문자열: 다음 글자 맞히기
chars = sorted(set(input_str + label_str))   # 문자 집합: ['!', 'a', 'e', 'l', 'p']
char2idx = {c: i for i, c in enumerate(chars)}
print("문자 집합:", chars)

x_idx = [char2idx[c] for c in input_str]     # 'apple' → [1, 4, 4, 3, 2]
y_idx = [char2idx[c] for c in label_str]
X = torch.eye(len(chars))[x_idx].unsqueeze(0)   # 원-핫 벡터로 바꾸고 배치 차원 추가: (1,
5, 5)
Y = torch.LongTensor(y_idx).unsqueeze(0)        # (1, 5)

class CharRNN(nn.Module):
    def __init__(self, size):
        super().__init__()
        self.rnn = nn.RNN(size, size, batch_first=True)
        self.fc = nn.Linear(size, size)          # 각 시점의 출력 → 다음 글자 점수
    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(out)

model = CharRNN(len(chars))
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)

for epoch in range(100):
    pred = model(X)                              # (1, 5, 5): 시점마다 5개 글자의 점수
    loss = loss_fn(pred.view(-1, len(chars)), Y.view(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch in (0, 5, 10, 99):
        guess = "".join(chars[i] for i in pred.argmax(dim=2)[0])
        print(f"Epoch {epoch:3d}  Loss: {loss.item():.4f}  예측: {guess}")
```

실행 결과

```
문자 집합: ['!', 'a', 'e', 'l', 'p']
Epoch   0  Loss: 1.6441  예측: lllll
Epoch   5  Loss: 0.8817  예측: pple!
Epoch  10  Loss: 0.2013  예측: pple!
Epoch  99  Loss: 0.0004  예측: pple!
```

직접 해 보기 `input_str, label_str`를 `"hello", "ello!"`로 바꿔 보세요. 두 번째 l 다음에 o가 나오려면 은닉 상태가 무엇을 기억해야 할까요?

실습 7 시점마다 원-핫 → 은닉 상태 → 점수 → 확률 따라가기

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
input_str, label_str = "apple", "pple!"
chars = sorted(set(input_str + label_str))       # ['!', 'a', 'e', 'l', 'p']
char2idx = {c: i for i, c in enumerate(chars)}
X = torch.eye(len(chars))[[char2idx[c] for c in input_str]].unsqueeze(0)
Y = torch.LongTensor([[char2idx[c] for c in label_str]])

class CharRNN(nn.Module):
    def __init__(self, size):
        super().__init__()
        self.rnn = nn.RNN(size, size, batch_first=True)
        self.fc = nn.Linear(size, size)
    def forward(self, x):
        h, _ = self.rnn(x)
        return h, self.fc(h)
```

```
model = CharRNN(len(chars))
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)
for epoch in range(100):                              # 07-03 실습과 같은 학습
    _, scores = model(X)
    loss = nn.functional.cross_entropy(scores.view(-1, len(chars)), Y.view(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# 학습된 모델에 'apple'을 넣고, 시점마다 벡터가 어떻게 바뀌는지 따라가기
fmt = lambda v: "[" + ", ".join(f"{x:5.2f}" for x in v.tolist()) + "]"
print("문자 순서:", chars)
with torch.no_grad():
    h, scores = model(X)
    probs = torch.softmax(scores, dim=2)
for t, c in enumerate(input_str):
    print(f"\n시점 {t + 1}: 입력 '{c}'")
    print("  원-핫 x_t      ", fmt(X[0, t]))
    print("  은닉 상태 h_t  ", fmt(h[0, t]), " ← tanh(W_x·x_t + W_h·h_(t-1) + b)")
    print("  점수 W_y·h_t+b ", fmt(scores[0, t]), " ← nn.Linear")
    print("  소프트맥스 확률", fmt(probs[0, t]), f" → 예측 '{chars[probs[0, t].argmax
()]}' (정답 '{label_str[t]}')")
```

실행 결과

```
문자 순서: ['!', 'a', 'e', 'l', 'p']

시점 1: 입력 'a'
  원-핫 x_t       [ 0.00,  1.00,  0.00,  0.00,  0.00]
  은닉 상태 h_t   [-0.91,  0.87,  0.99, -0.87,  0.91]  ← tanh(W_x·x_t + W_h·h_(t-1) + b)
  점수 W_y·h_t+b  [-3.91, -2.77,  0.91, -4.24,  9.01]  ← nn.Linear
  소프트맥스 확률 [ 0.00,  0.00,  0.00,  0.00,  1.00]  → 예측 'p' (정답 'p')

시점 2: 입력 'p'
  원-핫 x_t       [ 0.00,  0.00,  0.00,  0.00,  1.00]
  은닉 상태 h_t   [-0.97,  0.97,  0.30, -1.00, -1.00]  ← tanh(W_x·x_t + W_h·h_(t-1) + b)
  점수 W_y·h_t+b  [-0.89, -3.17, -2.91, -0.11,  8.98]  ← nn.Linear
  소프트맥스 확률 [ 0.00,  0.00,  0.00,  0.00,  1.00]  → 예측 'p' (정답 'p')

시점 3: 입력 'p'
  원-핫 x_t       [ 0.00,  0.00,  0.00,  0.00,  1.00]
  은닉 상태 h_t   [ 0.92, -1.00, -0.70, -1.00, -1.00]  ← tanh(W_x·x_t + W_h·h_(t-1) + b)
  점수 W_y·h_t+b  [-1.49, -2.82, -1.38,  7.24, -2.05]  ← nn.Linear
  소프트맥스 확률 [ 0.00,  0.00,  0.00,  1.00,  0.00]  → 예측 'l' (정답 'l')

시점 4: 입력 'l'
  원-핫 x_t       [ 0.00,  0.00,  0.00,  1.00,  0.00]
  은닉 상태 h_t   [-0.82, -1.00, -0.98, -0.17,  1.00]  ← tanh(W_x·x_t + W_h·h_(t-1) + b)
  점수 W_y·h_t+b  [-2.15, -2.30,  7.35, -2.03, -1.81]  ← nn.Linear
  소프트맥스 확률 [ 0.00,  0.00,  1.00,  0.00,  0.00]  → 예측 'e' (정답 'e')

시점 5: 입력 'e'
  원-핫 x_t       [ 0.00,  0.00,  1.00,  0.00,  0.00]
  은닉 상태 h_t   [ 0.41,  1.00, -0.99,  1.00, -0.75]  ← tanh(W_x·x_t + W_h·h_(t-1) + b)
  점수 W_y·h_t+b  [ 8.03, -2.46, -1.52, -0.10, -1.38]  ← nn.Linear
  소프트맥스 확률 [ 1.00,  0.00,  0.00,  0.00,  0.00]  → 예측 '!' (정답 '!')
```

직접 해 보기 시점 2와 3의 원-핫 입력은 같습니다. 그런데 은닉 상태가 다른 이유를 정의의 $\tanh(W_x x_t + W_h h_{t-1} + b)$에서 찾아보세요.

07-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 Char RNN은 시점마다 다음 글자를 맞히도록 학습한다. (O / X) 이름 _____

2 Char RNN의 입력과 정답은 같은 문자열이다. (O / X) 이름 _____

3 문자를 신경망에 넣기 전에 바꾸는 형태는? 이름 _____

① 원-핫 벡터 ② 이미지 ③ 정답 레이블

4 같은 "p" 입력에 다른 다음 글자를 낼 수 있는 이유는? 이름 _____

① 은닉 상태에 앞 문맥이 담겨 있어서 ② 가중치가 시점마다 달라서 ③ 문자 집합이 커서

5 실습의 문자 집합에는 _____종류의 문자가 있다. 이름 _____

6 Char RNN은 입출력 구성 중 다 대 _____ 구조다. 이름 _____

7 문자 단위 RNN을 정의하고, 휴대폰 키보드 추천에 적용해 설명해 보세요. 이름 _____

원문: 07-03 문자 단위 RNN — https://wikidocs.net/64703

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
시퀀스 / 시점	순서가 의미를 가지는 데이터 / 몇 번째 원소인지	"나는 오늘 밥을 먹었다" = 시점 4개
피드 포워드의 한계	입력 크기 고정, 이전 입력에 대한 기억 없음	단어 수가 다른 문장마다 다른 모델 필요
은닉 상태	$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$, 가중치는 시점끼리 공유	같은 "먹었다"라도 앞 문맥에 따라 다른 상태
장기 의존성 문제	길면 앞쪽 시점의 기울기가 사라져 앞 정보를 잊음	50개 뒤 맨 앞 영향: RNN 2.4×10^{-11}
LSTM	셀 상태 + 입력·삭제·출력 게이트로 기억 관리	"파리"를 저장·유지했다가 "프랑스어"에 사용
GRU	업데이트·리셋 게이트 2개, 셀 상태 없음	파라미터 LSTM 480 vs GRU 360
Char RNN	문자 단위 다 대 다 RNN, 시점마다 다음 문자 예측	apple → pple!

다음

다음 자료 예고: 08장 합성곱 신경망

RNN이 "순서"를 다루는 신경망이라면, 합성곱 신경망(CNN)은 "주변 이웃"을 함께 보는 신경망입니다. 이미지를 위해 만들어졌지만, 13장에서 문장 분류에도 쓰입니다.

정답과 해설

07-01 순환 신경망(RNN)

1. O
2. X — 모든 시점이 같은 가중치를 공유합니다.
3. ②
4. ②
5. 시점(time step)
6. 문맥
7. 예시 답: 은닉 상태는 지금까지 읽은 내용의 요약으로, 현재 입력과 이전 은닉 상태로 갱신된다. "나는"을 읽고 만든 기억에 "오늘", "밥을", "먹었다"가 차례로 더해지며 갱신된다.

07-02 LSTM과 GRU

1. O
2. X — LSTM은 3개, GRU는 2개입니다.
3. ①
4. ① — 삭제 게이트가 0이면 이전 장기 기억을 지웁니다.
5. 장기 의존성
6. 0.23 (2.3×10^{-1})
7. 예시 답: 입력 게이트는 새 정보 저장, 삭제 게이트는 옛 기억 유지/삭제, 출력 게이트는 꺼내 쓸 기억 선택. "파리"를 읽을 때 입력 게이트로 저장하고, 중간에는 삭제 게이트로 유지하고, 빈칸에서 출력 게이트로 꺼내 "프랑스어"를 맞힌다.

07-03 문자 단위 RNN

1. O
2. X — 정답은 입력을 한 글자씩 민 문자열입니다.
3. ①
4. ①
5. 5
6. 다
7. 예시 답: 문자 단위로 입력을 받아 시점마다 다음 문자를 예측하는 RNN이다. 키보드가 지금까지 친 글자를 보고 다음 글자를 추천하는 것과 같다.

용어 정리

용어	정의
시퀀스 데이터	순서가 의미를 가지는 데이터 (문장, 음성, 주가)
시점 (time step)	시퀀스에서 몇 번째 원소인지
피드 포워드 신경망	입력에서 출력 방향으로 한 번만 계산하는 신경망 (MLP, CNN)
메모리 셀 / 은닉 상태	RNN의 계산 단위 / 셀이 다음 시점으로 넘기는 기억 h_t
깊은 RNN / 양방향 RNN	RNN 층을 여러 개 쌓은 것 / 앞→뒤, 뒤→앞으로 함께 읽는 것
바닐라 RNN	게이트 없는 가장 기본적인 RNN
장기 의존성 문제	시퀀스가 길면 앞쪽 정보가 뒤까지 전달되지 못하는 문제
LSTM / 셀 상태	게이트로 기억을 관리하는 RNN / LSTM의 장기 기억 C_t
게이트	시그모이드로 0~1 값을 만들어 정보를 얼마나 통과시킬지 정하는 장치
GRU	업데이트·리셋 게이트 2개로 단순화한 LSTM
Char RNN	문자 단위로 다음 문자를 예측하는 RNN